American Terawatt

DEVELOPER INFRASTRUCTURE

Local LLM on the Shared GPU Pool

Claiming GPUs, serving an inference endpoint and answering from our own documents

End-to-end account of how a local large language model is deployed on the shared GPU pool — how GPUs are borrowed, why the work happens inside a container, where files must live to survive, how the model runtime is installed into an image without package manager access, and how the resulting endpoint is reached from a remote machine — followed by *corpus*, the retrieval system that answers questions from our own documents over that endpoint.

Revision history

REV.	DATE	AUTHOR	DESCRIPTION
0.1	Sep. 17, 2026	Eugen Starschich	Initial document
0.2	Sep. 17, 2026	Eugen Starschich	*corpus* document retrieval added (Sections 7 and 8); command reference extended
0.3	Sep. 17, 2026	Eugen Starschich	Model call sites tabulated (Section 7.4); embedding and chat endpoints distinguished
0.4	Sep. 18, 2026	Eugen Starschich	Main model changed to `qwen3.5:122b-a10b`; capability matrix and head-to-head measurements added (Section 5); agent integration added (Section 9); four pitfalls added

CONTENT

1 Architecture 4
1.1 Scope 4
1.2 Hosts involved 4

2 Borrowing GPUs 5
2.1 The verbs 5
2.2 What a claim actually creates 5
2.3 Why a container at all 5
2.4 Consequences to plan around 6

3 Storage: what survives 7

4 Installing the runtime 8
4.1 Staging the binary 8
4.2 Where Ollama sits 8

5 Fetching and serving the model 9
5.1 Start the server 9
5.2 Pull the weights 9
5.3 Model inventory 9
5.4 Capabilities are not interchangeable 9
5.5 Measured behaviour 10

6 Remote access 12

7 Document retrieval on the endpoint: *corpus* 13
7.1 What it is 13
7.2 Where everything runs 13
7.3 Ingest: documents in, index out 13
7.4 Query: three model calls, not one 14

8 Operating *corpus* 16
8.1 Four ways in, one core 16
8.2 Everyday commands 16
8.3 Known limits 17
8.4 Backups 17

9 Connecting an agent 18
9.1 Three things an agent needs 18
9.2 The provider must be named 18
9.3 Auxiliary slots 19
9.4 Wiring in the corpus 19
9.5 Verifying the wiring 19

10 Teardown and restore 21

11 Pitfalls 22

12 Command reference 23

1 Architecture

1.1 Scope

This document records a working deployment of `qwen3.5:122b-a10b` under Ollama on one NVIDIA RTX PRO 6000 Blackwell card of the shared GPU pool, with `gpt-oss:120b` retained as a faster fallback without vision. All concrete figures quoted in Section 5 come from that deployment, and Section 9 records how an agent is pointed at it.

1.2 Hosts involved

Four hosts participate. Nothing about the model runs on the developer VM; it is a control point and an SSH waypoint only.

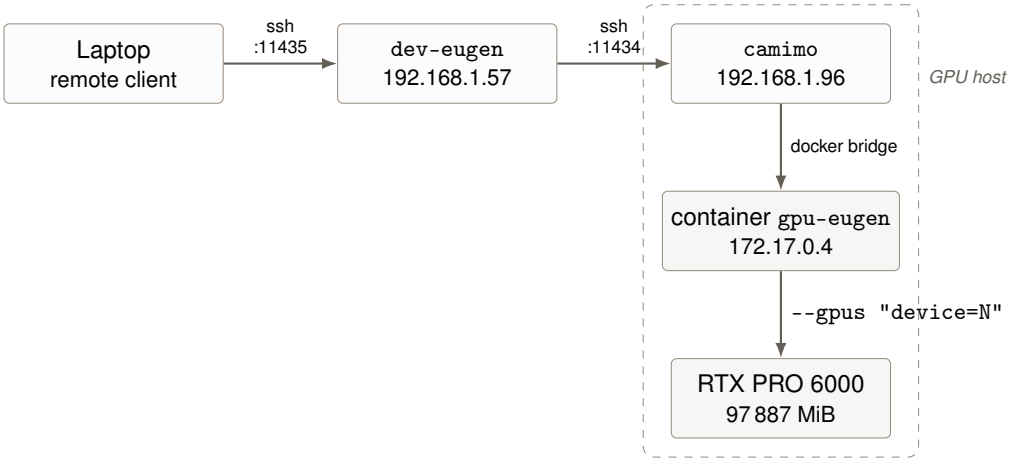


Figure 1: Hosts, container and card, with the two SSH hops that carry the endpoint out to the client.

Host	Role	Notes
Laptop	remote client	reaches the pool only via SSH
`dev-eugen`	developer VM	4 cores, 8 GB RAM, *no GPU*
`camimo`	GPU host	8 GPUs, also the NFS server for `/shared`
`gpu-eugen`	per-claim container	128 cores, 440 GB RAM visible

Table 1: The four hosts and what each contributes.

The GPU host and the storage host are the same machine: `/shared` is local disk there, and NFS-mounted on the developer VM.

AMERICA'S PRIVATE INDUSTRIAL POWER GRID

2 Borrowing GPUs

2.1 The verbs

```
gpu status        # who holds what, what is free
gpu claim 2       # take 2 (all-or-nothing: 2 or none)
gpu shell         # shell inside your container
gpu shell CMD     # run one command in it, non-interactively
gpu release       # give them back
```

`gpu` on the developer VM is a thin relay: it re-quotes its arguments and forwards them over SSH to `camimo`, where the real tool runs. Because it requests a remote TTY only when it has one, it works from scripts and from automated callers as well as interactively.

2.2 What a claim actually creates

`gpu claim` allocates cards in pool state and then starts a container pinned to exactly those cards:

```
docker run -d --name gpu-eugen \
  --gpus "device=1" \
  --label gpu.owner=eugen --label gpu.devices=1 \
  --restart no \
  --user 3003:3000 \
  -e HOME=/home/eugen \
  -v /shared:/shared \
  --shm-size 16g \
  nvidia/cuda:12.6.0-base-ubuntu24.04 sleep infinity
```

Pool state lives in `/var/lib/gpu-pool/state.json` under a `flock`, and every command reconciles that file against `docker ps` before reporting or allocating. A container that dies therefore returns its cards on its own. If the state file is unreadable the tool refuses to write rather than falling back to “the pool is empty”, which would hand out cards already in use.

2.3 Why a container at all

Property	Consequence
`--gpus "device=N"`	Isolation is structural, not by convention. Other cards are invisible; yours renumbers to 0. A stray `CUDA_VISIBLE_DEVICES` cannot reach a colleague’s run.
Container is the unit of accounting	`gpu release` is `docker rm -f`; reconciliation against `docker ps` frees the cards.
No reaper, by design	A claim is released by its owner, never by a timer — so a long training run is safe, and a forgotten claim blocks the pool.
All-or-nothing allocation	Asking for 4 with 3 free yields nothing, rather than a half-sized allocation that silently invalidates the run.
`--user uid:3000`	Files created on `/shared` are owned by you in the `team` group, not by root.

Table 2: What the container arrangement buys.

2.4 Consequences to plan around

The image is `nvidia/cuda:12.6.0-base-ubuntu24.04`. The *base* variant carries the CUDA runtime and nothing else.

Constraint	Why
No `python3`, `curl`, `git`, `tmux`	`-base` image variant
No `sudo`; `apt install` fails	runs as your uid, not root
Service ports unreachable from outside	default docker bridge, isolated netns
Filesystem is ephemeral	`gpu release` deletes the container

Table 3: Constraints imposed by the claim container.

Anything that must run inside the container therefore has to be a self-contained binary staged in from outside.

3 Storage: what survives

Exactly one path spans both machines.

Path	Shared with container?	Survives `gpu release`?
`/shared/$USER`	yes, live and bidirectional	yes
`~` on the developer VM	**no**	n/a
`~` inside the container	it is *camimo's* home	yes, on camimo
container root filesystem	—	no

Table 4: Which paths are shared and which survive a release.

This deserves emphasis because it is easy to get wrong. The claim command bind-mounts `-v $HOME:$HOME`, but `$HOME` is resolved *on the GPU host*. The home directory visible inside the container is camimo's `/home/$USER`, not the developer VM's. A file written to `~` in one place is not visible in the other. Only `-v /shared:/shared` is unconditional, and only `/shared` genuinely spans both.

`/shared` **is not a backup.** One unmirrored NVMe copied hourly to another machine. Model weights belong there because they are large and re-downloadable; anything whose loss would matter belongs in Git, pushed elsewhere. It is also readable by the whole team, so it must never hold credentials, keys, tokens, draft commercial terms, or material under NDA.

4 Installing the runtime

4.1 Staging the binary

Because the container has no downloader and no root, the runtime is fetched on the developer VM and staged through `/shared`.

```
# on the developer VM
mkdir -p /shared/$USER/opt /shared/$USER/ollama-models
cd /shared/$USER/opt
curl -sL -o ollama.tar.zst \
  https://github.com/ollama/ollama/releases/download/v0.34.0/ollama-linux-amd64.tar.zst
mkdir -p ollama && zstd -dc ollama.tar.zst | tar -xf - -C ollama
```

This yields `bin/ollama` plus the bundled CUDA runners in `lib/ollama`. No installation step, no systemd unit, no root required — which is precisely why it works in a container where none of those are available.

4.2 Where Ollama sits

Ollama is a management layer, not the inference engine and not the model.

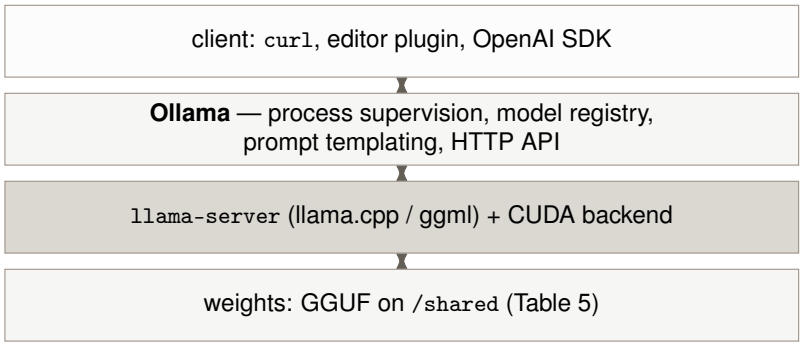


Figure 2: The serving stack, from client call down to the weights on disk.

Ollama supervises a separate `llama-server` process; llama.cpp and its CUDA kernels perform the arithmetic. Ollama contributes four things: it fetches and stores weights as blobs and manifests, it loads and unloads models with a keep-alive timer, it renders chat turns into the exact token format the model was trained on, and it exposes both a native API and an OpenAI-compatible one. It also sizes the context window to measured VRAM at start-up.

For a single user on a single card this is the right trade. For a shared, heavily concurrent endpoint, vLLM's continuous batching would serve better at the cost of a much heavier install.

5 Fetching and serving the model

5.1 Start the server

Keep this in a script on `/shared`. Long quoted one-liners do not survive the relay's re-quoting.

```
#!/bin/bash
# /shared/$USER/start-ollama.sh   --  run: gpu shell /shared/$USER/start-ollama.sh
set -u
BIN=/shared/$USER/opt/ollama/bin/ollama
LOG=/shared/$USER/ollama-serve.log

export OLLAMA_MODELS=/shared/$USER/ollama-models
export OLLAMA_HOST=0.0.0.0:11434

if ps -eo cmd | grep -q "[b]in/ollama serve"; then
    echo "already running"; exit 0
fi
cd /shared/$USER
nohup setsid "$BIN" serve > "$LOG" 2>&1 < /dev/null &
echo "launched pid $!"
```

5.2 Pull the weights

```
gpu shell bash -lc 'export OLLAMA_MODELS=/shared/$USER/ollama-models; \
  /shared/$USER/opt/ollama/bin/ollama pull qwen3.5:122b-a10b'
```

Because `OLLAMA_MODELS` points at `/shared`, the pull happens once and is reused by every later claim — and by teammates. Sustained download rate is around 110–120 MB/s, though the tail of a large blob has been observed to collapse to roughly 1 MB/s for the final gigabyte; the pull still completes.

5.3 Model inventory

Three models are resident on `/shared`, serving distinct roles.

Model	Size	Role
`qwen3.5:122b-a10b`	81.4 GB	Main model. 122 B total / 10 B active sparse MoE. Declares *completion, vision, tools, thinking*.
`gpt-oss:120b`	65.4 GB	Fallback. Faster, but no vision.
`bge-m3:latest`	1.2 GB	Embeddings for *corpus* (Section 7). Not a chat model.

Table 5: Model store on `/shared/$USER/ollama-models`, 147.9 GB total.

5.4 Capabilities are not interchangeable

A model's declared capabilities decide what it can be used *for*, and no configuration can add a missing one. Query them before wiring a model into anything:

```
curl -s http://localhost:11434/api/show -d '{"model":"qwen3.5:122b-a10b"}' \
  | python3 -c "import json,sys; print(json.load(sys.stdin)['capabilities'])"
```

An agent framework attaches a `tools` array to every request. Sending that to a model lacking the capability is rejected outright:

Model	completion	tools	vision	thinking
`qwen3.5:122b-a10b`	yes	yes	yes	yes
`gpt-oss:120b`	yes	yes	**no**	yes
`bge-m3:latest`		*embedding only*		

Table 6: Declared capabilities. Two consequences follow directly: a model without *tools* cannot drive an agent, and a model without *vision* cannot read an image.

```
HTTP 400
{"error":{"message":"registry.ollama.ai/library/<model> does not support tools",
          "type":"invalid_request_error"}}
```

The same model answers normally when the array is absent, so the model is not faulty — it simply cannot participate in an agent loop.

5.5 Measured behaviour

Both chat models were put through an identical battery on the same endpoint: the same prompts, the same figure, and the same scoring. Everything in Table 7 is measured, not quoted from a model card.

Quantity	`gpt-oss:120b`	`qwen3.5:122b-a10b`
Size on disk	65.4 GB	81.4 GB
VRAM resident	67.0 GB	84.9 GB
Context served	131 072	262 144
Cold load and first reply	43 s	28.6 s
Generation, warm	**183.5** tok/s	126.8 tok/s
Tokens to emit “CONNECTION OK”	**68**	253
Tool call, first turn	0.8 s	1.0 s
Edit call, second turn	1.1 s	3.7 s
`old_string` byte-exact	yes	yes
Edited code correct	yes	yes
Vision	*not supported*	**yes**
Figure → TikZ compiles	—	yes
Figure → TikZ, SSIM	—	0.9441
Figure → TikZ, ink overlap	—	0.4281

Table 7: Head-to-head on one RTX PRO 6000. Keep-alive on both is 5 minutes idle by configuration, though a longer effective value has been observed.

Reading the table. `gpt-oss:120b` generates about 45 % faster and is markedly less verbose — 68 completion tokens to answer a two-word prompt against 253. In an agent loop that difference compounds across every turn, in both latency and context consumed.

`qwen3.5:122b-a10b` pays for that with vision and twice the context, and matches `gpt-oss` exactly where it matters most for agent work: both produced a byte-exact `old_string` for a search-and-replace edit and both yielded correct, compiling code. Byte-exactness is the usual failure mode for weaker models — a paraphrased original silently fails to apply.

On the figure-to-TikZ task its ink overlap of 0.4281 is more than four times that of the dedicated vision model it replaced (`qwen2.5vl:32b`, 0.1019), though still below the deterministic converter’s 0.7015. The model’s output is semantic and editable; the converter’s is a coordinate dump.

VRAM constrains the context window. With `qwen3.5:122b-a10b` resident, `nvidia-smi` reports 84 952 MiB of 97 887 in use — roughly 13 GB free. The KV cache for a 256 K context on a 122 B model does not fit in that, so a client configured for the model's full native context will fail on allocation during long conversations or large retrievals. Configure clients for 131 072 on a single card, or claim a second card (Section 2) before raising it.

6 Remote access

The server binds inside the container's network namespace, which nothing outside can reach. Two SSH forwards carry it out. Every hop terminates on loopback, so no inference port is ever published to the office network.

```
# 1. in the container: bind the bridge interface
gpu shell /shared/$USER/start-ollama.sh

# 2. on the developer VM: forward to the container
ssh -f -N -L 127.0.0.1:11434:172.17.0.4:11434 camimo

# 3. on the laptop: forward to the developer VM
ssh -N -L 11435:127.0.0.1:11434 eugen@192.168.1.57

# 4. verify, from a second terminal
curl http://localhost:11435/api/tags
```

The client then uses `http://localhost:11435/v1` as an OpenAI-compatible base URL with any non-empty API key, or `/api/...` for the native interface.

Security. Ollama performs no authentication whatsoever. Whoever reaches the port can run inference on the card and pull or delete models. Binding `0.0.0.0` inside the container exposes it to every account on the GPU host — acceptable within a team, but a deliberate decision rather than a default. Binding `0.0.0.0` on the developer VM would publish an unauthenticated endpoint to the whole LAN and should not be done. Access from outside the office requires a VPN; the developer VM has no public address.

7 Document retrieval on the endpoint: *corpus*

7.1 What it is

corpus is a retrieval-augmented question answering system over our own documents, running entirely on American Terawatt hardware and using the endpoint of Section 6 as its model. Documents are parsed once, split into passages, embedded, and indexed on local disk. At question time the relevant passages are retrieved and handed to `gpt-oss:120b` on the shared GPU pool, which answers from those passages and cites them. Nothing leaves the network, and nothing is sent to an external API.

The system deliberately does *not* fine-tune a model on the documents. Fine-tuning gives no reliable recall of specific facts, no citations, and would need re-running whenever a document changes. Retrieval at query time gives all three.

7.2 Where everything runs

Three machines are involved, and the split matters: the corpus lives next to the documents, the model lives on the GPU, and only questions and answers cross the network.

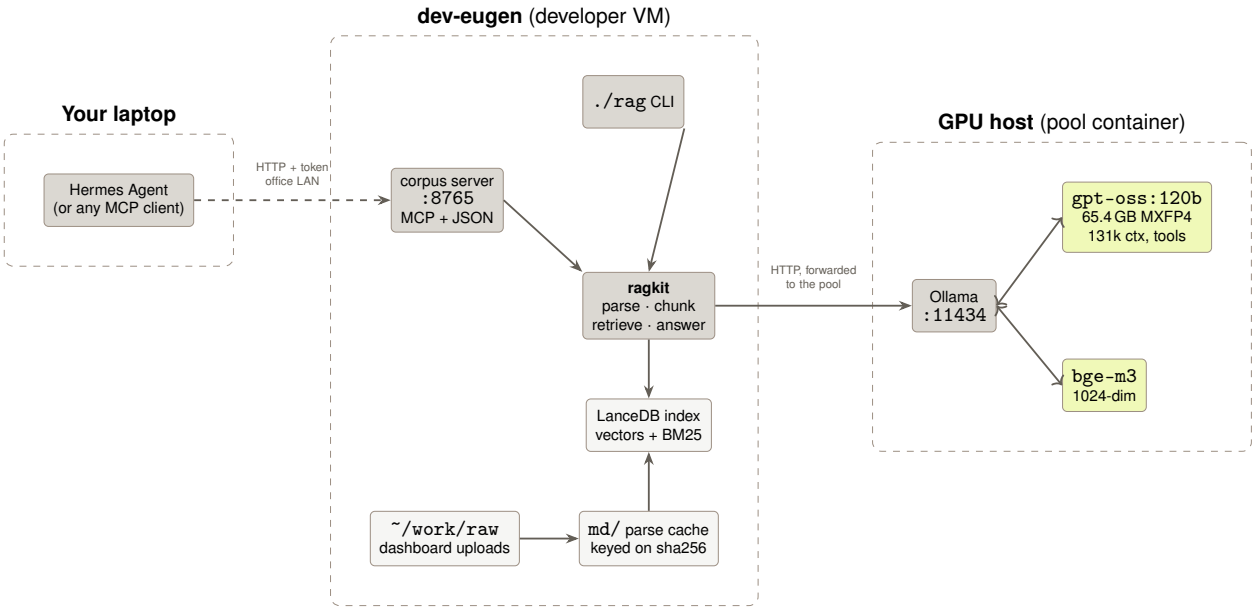


Figure 3: Where everything runs. Only questions and answers cross the network; the index stays beside the documents.

Two consequences are worth holding onto. First, **the corpus is not the model**: the laptop's SSH tunnel of Section 6 reaches the model on port 11434, but the corpus is a separate service on port 8765 and needs its own path. Second, **retrieval happens where the data is**. Only a question goes over the wire and only an answer comes back — kilobytes, not the gigabytes it would take to replicate the index.

7.3 Ingest: documents in, index out

Ingest runs once per document, and is cached. Parsing is the slow step, so its output is stored in `md/` keyed on the source file's sha256 — re-running `./rag ingest` over a growing folder only touches files whose bytes changed.

Why the chunks look the way they do

Every chunk is prefixed with its document title and heading path before it is embedded. A passage reading "the limit is 40 °C" is useless on its own; "TB 754 $>$ 6. Harmonic limits $>$ 6.11 — the limit is 40 °C" both

AMERICA'S PRIVATE INDUSTRIAL POWER GRID

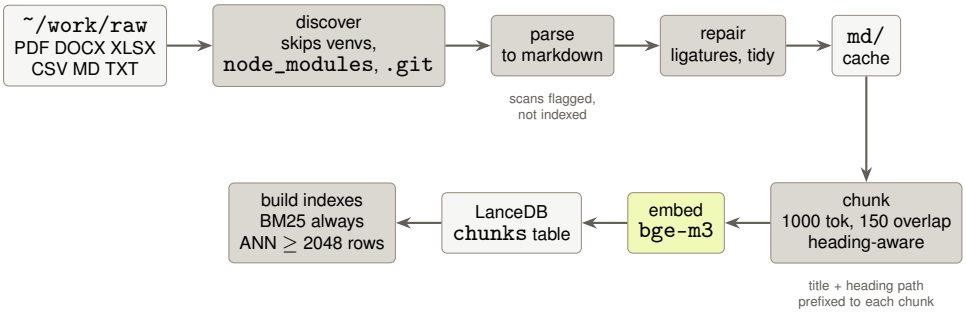


Figure 4: Ingest. Parsing is cached on the source's sha256, so re-running only touches documents whose bytes changed.

retrieves and answers correctly. The prefix is what gets embedded and BM25-indexed; the bare text is what the model is shown.

Tables are kept whole where possible, and split on row boundaries with the header repeated when they are too large — a spreadsheet row severed from its column headings cannot be read by anything.

7.4 Query: three model calls, not one

A question triggers an embedding call, then a rerank call, then the answer call. The database and the model never talk to each other; Python moves text between them.

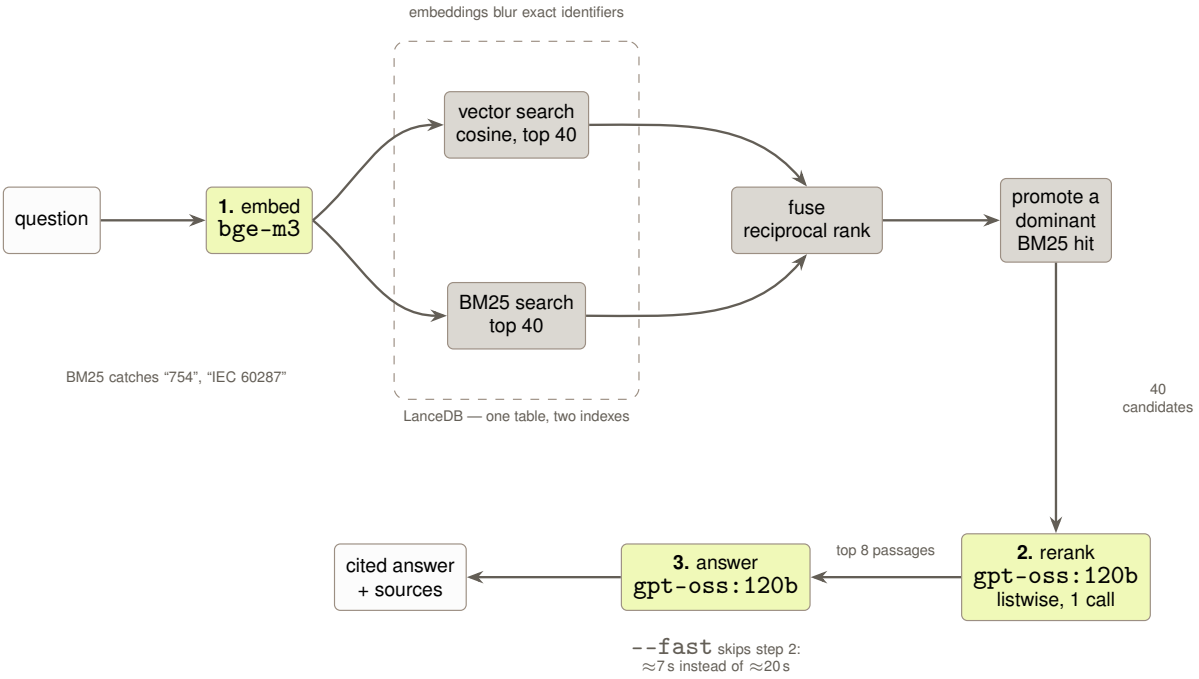


Figure 5: A query. Three model calls: embed the question, rerank the shortlist, then answer from the survivors.

Which model does what

There are exactly four model calls in the whole system, and **ragkit** makes all of them. The two models are not interchangeable: `bge-m3` only turns text into vectors — it cannot generate — so anything that produces words is `gpt-oss:120b`.

`llm.embed` posts to `/api/embed`, `llm.chat` to `/v1/chat/completions` — same Ollama, same connection, different model name in the payload. That is why Figure 3 shows one arrow into Ollama and Ollama reaching both models.

Call site	Wrapper	Model	When
`rag.py:87`	`llm.embed`	`bge-m3`	ingest: embed every chunk
`retrieve.py:62`	`llm.embed`	`bge-m3`	query: embed the question
`retrieve.py:192`	`llm.chat`	`gpt-oss:120b`	rerank the shortlist
`answer.py:36`	`llm.chat`	`gpt-oss:120b`	write the cited answer

Table 8: The four model calls, and which model serves each.

Where the LanceDB search sits in that list is: nowhere. The lookup itself (vector + BM25) uses no model at all. It is the steps on either side of it that do — embedding the question before, reranking after. Likewise "the database and the model never talk to each other" means exactly that: LanceDB and `gpt-oss:120b` have no link, and **ragkit** is the thing in the middle that talks to both.

How the model actually receives the passages

As plain text in the prompt. There is no special channel and nothing is loaded into the model. The retrieved passages are concatenated into one numbered block and sent as an ordinary chat message to `/v1/chat/completions`:

system Use only the numbered sources. Cite as [3]. If the sources do not contain the answer, say so.
user Sources:
[1] 0754 AC side harmonics > 6. Harmonic limits > 6.11.2 . . .
distortion factor of the centred interharmonic subgroup . . .
[2] . . .
Question: What harmonic order range does brochure 754 consider?

The model sees one flat sequence of tokens. It has no filesystem access, no connection to LanceDB, and no memory of previous questions — each question builds a fresh prompt. Three things follow:

- **Citations cannot drift.** The model copies back the `[n]` label it was given; the mapping from `[n]` to a file path is done in Python afterwards.
- **Refusal is honest.** "The sources do not contain any entry for brochure 9999" is an accurate report about its input.
- **Retrieval is the ceiling.** If a passage does not reach the prompt, the model cannot know it, however capable it is.

Why context size is not the constraint

	Tokens
System message	128
Retrieved passages (4) + question	4 103
Typical query at `k=8`	≈8 000
One brochure in full (TB 754)	74 550
Model context window	131 072

Table 9: Token budget of a query against the context window.

A default query uses about 6 % of the window. Stuffing the context instead would fit one brochure and a half out of roughly a thousand. That is the whole argument for retrieval: the problem was never how much text fits, it is picking the right four thousand tokens.

8 Operating *corpus*

8.1 Four ways in, one core

The rag script used in the following commands is located in:

`/home/eugen/work/corpus/rag`

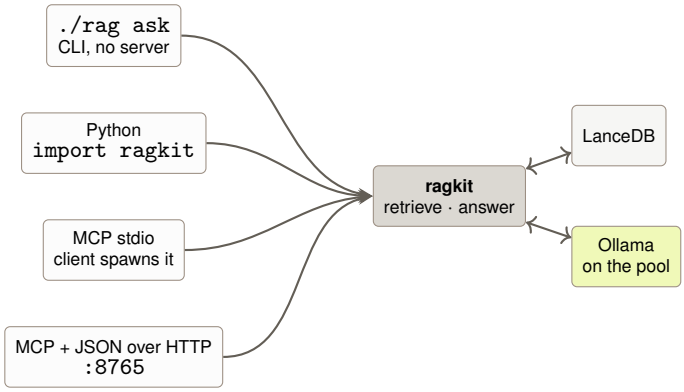


Figure 6: Four interfaces onto one pipeline — the same **ragkit** core backs every one of them.

MCP is a doorway for other programs, not part of retrieval. Working on the developer VM, it is never needed. It exists so an agent on another machine — Hermes, say — can use the corpus, and so a coding session can search the documents mid-conversation.

Interface	Server?	Auth	Use it for
`./rag ask`	no	—	everyday use on the developer VM
`import ragkit`	no	—	notebooks, own scripts
MCP stdio	spawned	—	Claude Code on that box
MCP `/mcp`	yes	bearer	Hermes and other agents
JSON `/ask`, `/search`	yes	bearer	curl, scripts, any language

Table 10: The four interfaces and what each is for.

8.2 Everyday commands

Task	Command
Index new uploads	`./rag ingest`
Index elsewhere	`./rag ingest /shared/eugen/brochures`
Re-parse everything	`./rag ingest --force`
Rebuild from scratch	`./rag ingest --reset`
Ask	`./rag ask "..."`
Passages only	`./rag search "..."`
What is indexed	`./rag status`
Start the server	`tmux new -s corpus-mcp`, then `./mcp-http --host 0.0.0.0`
Client details	`./mcp-http --show-client-config`

Table 11: Everyday *corpus* commands on the developer VM.

Documents reach the developer VM through the **dashboard upload panel** at

`http://dev.americanterawatt.com/`

landing in `~/work/raw`. That is the route; there is no SSH access for anyone, by design.

8.3 Known limits

- **Scanned PDFs index nothing.** Ingest flags them rather than silently adding empty chunks. The main model (`qwen3.5:122b-a10b`) has vision and is the obvious OCR path if needed. The dedicated vision model previously noted here, `qwen2.5vl:32b`, was removed once the main model superseded it.
- **Damage already in the source is not repaired.** The eCIGRE brochure list contains titles that lost their ligatures before reaching the spreadsheet ("Model specication"), so a search for "specification" will not match them. Ligature repair runs only for PDFs, where the document's own text supplies a trustworthy vocabulary.
- **One writer.** LanceDB is embedded: concurrent readers are fine, two simultaneous ingests are not.
- **Rerank latency.** The 120B spends reasoning tokens before any visible output, so a reranked query costs ≈20 s.
- **LAN exposure.** With `--host 0.0.0.0` the bearer token is the only thing between the office network and every indexed document.

8.4 Backups

`index/` and `md/` are derived — they can be deleted and rebuilt from the sources at any time. The pipeline code is what matters, and as Section 3 notes `/shared` is one unmirrored NVMe, so that directory belongs in Git, pushed elsewhere.

9 Connecting an agent

The endpoint of Section 6 and the retrieval tools of Section 8 are both consumed by the same client: an agent that reasons, calls tools, and reads our documents. This section records how that client is configured, using Hermes Agent as the worked example. The principles carry to any agent framework; the key names do not.

9.1 Three things an agent needs

1. A model that declares *tools* — see Section 5.4. Without it the agent cannot act at all.
2. A provider entry that actually resolves to our endpoint.
3. The MCP server, for retrieval over our own documents.

9.2 The provider must be named

A custom OpenAI-compatible endpoint is declared as a provider. In Hermes Agent a *bare* `provider: custom` does not work: the resolver returns `None` for the literal string `"custom"`,

```
if not requested_norm or requested_norm == "custom": return None
```

and resolution falls through to the tool's default cloud provider, which is then sent our local model name together with whatever key is on file. The symptom is a complaint about a provider nobody configured — a rejected API key for a service that was never meant to be used — and nothing in the error indicates that a different endpoint was contacted. The remedy is a **named** provider entry plus `custom:<name>`, which gives resolution a real base URL to find.

```
providers:
  local:
    base_url: http://localhost:11435/v1     # the tunnel, not 11434
    api_key: no-key-required                # Ollama ignores it; must be non-empty
    default_model: "qwen3.5:122b-a10b"

model:
  default: "qwen3.5:122b-a10b"
  provider: custom:local                    # NOT a bare "custom"
  base_url: http://localhost:11435/v1
  context_length: 131072                    # see Section 5.5 on VRAM headroom
  max_tokens: 8192
  timeout_seconds: 300
```

Three of those values are worth justifying.

Port 11435, not 11434. On a laptop already running its own Ollama, the forward cannot bind 11434 and the client silently talks to the local instance instead — appearing to work, while using an entirely different model.

`context_length: 131072`**, not the model's native 262 144.** The KV cache does not fit the remaining VRAM on a single card (Section 5.5).

`max_tokens: 8192`**.** A reasoning model spends tokens before emitting anything visible; a tight budget returns empty content, which is indistinguishable from a hung request. A tool-calling turn that also writes an answer from retrieved passages needs the headroom.

9.3 Auxiliary slots

Where the main model lacks a capability, an agent may route that task to a separate model. Hermes calls these *auxiliary* slots — vision, web summarisation and compression among them — and, importantly, calls them **without** a tools array. That makes an auxiliary slot the correct home for a vision-only model, which would otherwise be rejected outright (Section 5.4).

```
auxiliary:
  vision:
    model: ""            # empty + "auto" = use the main chat model
    provider: "auto"
    base_url: null
```

With `qwen3.5:122b-a10b` as the main model the slot is left on `auto`, because that model has vision of its own. The slot earns its keep only when the main model does not — pairing `gpt-oss:120b`, which has tools but no vision, with a vision model that has no tools. Each covers the other's gap, and neither is asked to do what it cannot.

9.4 Wiring in the corpus

The MCP server is declared separately from the model, and is reached **directly over the office LAN** rather than through the SSH tunnel — it is a different service on a different port.

```
mcp_servers:
  corpus:
    url: http://192.168.1.57:8765/mcp
    headers:
      Authorization: "Bearer ${env:CORPUS_MCP_TOKEN}"
    timeout: 300          # rerank with the 120B takes ~20 s; 300 is the default
    connect_timeout: 60
    enabled: true
    tools:
      include: [search_corpus, ask_corpus, corpus_status]
      exclude: []
      resources: false
      prompts: false
```

The transport key is omitted deliberately: it is inferred from the presence of `url`, and is needed only to force SSE instead of Streamable HTTP. The token belongs in the agent's environment file, not here, so that the configuration itself stays safe to copy and to commit.

9.5 Verifying the wiring

Check the two halves independently before blaming the agent. The endpoint:

```
curl http://localhost:11435/api/tags          # must list the expected model
```

and the MCP server, which needs a session before it will answer — a bare `tools/list` returns `-32600 Missing session ID`:

```
TOK=$(ssh eugen@192.168.1.57 'cat ~/.config/corpus/mcp-token')
H=(-H "Authorization: Bearer $TOK" -H "Content-Type: application/json"
   -H "Accept: application/json, text/event-stream")

SID=$(curl -s -D- -X POST http://192.168.1.57:8765/mcp "${H[@]}" \
  -d '{"jsonrpc":"2.0","id":1,"method":"initialize","params":{
```

AMERICA'S PRIVATE INDUSTRIAL POWER GRID

```
       "protocolVersion":"2025-06-18","capabilities":{},
       "clientInfo":{"name":"healthcheck","version":"1"}}}' \
  -o /dev/null | tr -d '\r' | awk -F': ' '/^mcp-session-id/{print $2}')

curl -s -X POST http://192.168.1.57:8765/mcp "${H[@]}" \
  -H "mcp-session-id: $SID" \
  -d '{"jsonrpc":"2.0","method":"notifications/initialized"}' -o /dev/null

curl -s -X POST http://192.168.1.57:8765/mcp "${H[@]}" \
  -H "mcp-session-id: $SID" \
  -d '{"jsonrpc":"2.0","id":2,"method":"tools/list"}'
```

A request without the bearer token returns HTTP 401, which is the intended behaviour and a quick confirmation that authentication is enforced.

10 Teardown and restore

```
gpu release        # deletes the container; cards return to the pool
```

The container, the running server and both tunnels are destroyed. The Ollama installation, the model weights and the start-up script all live on `/shared` and survive. Restoring service after a later claim costs two commands and no re-download:

```
gpu claim 1
gpu shell /shared/$USER/start-ollama.sh
ssh -f -N -L 127.0.0.1:11434:172.17.0.4:11434 camimo
```

Confirm the bridge address first, as it may differ between claims:

```
ssh camimo "docker inspect -f \
  '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' gpu-eugen"
```

11 Pitfalls

Symptom	Cause and remedy
Claim never released	Claims do not expire, by design. A forgotten claim blocks the pool indefinitely. Release explicitly.
`Text file busy` on a freshly staged binary	NFS write-back lag on a new executable under `/shared`. It clears by itself; wait rather than re-copying.
`pgrep -f "ollama serve"` always matches	The search pattern appears in the searching command's own command line. Use `grep "[b]in/ollama serve"`.
Quoted one-liner silently does nothing	The relay re-quotes arguments; nested quoting does not survive. Put the commands in a script on `/shared`.
`curl` returns the wrong model	A local Ollama already occupies port 11434, so the forward failed to bind and the query answered locally. Forward to a free local port such as 11435.
`ssh -N` appears to hang	Expected. `-N` runs no remote command; it holds the tunnel open and prints nothing. Test from a second terminal.
Work lost on release	Only `/shared` spans both machines. The container home is the GPU host's, not the developer VM's.
`<defunct>` processes accumulate	Container PID 1 is `sleep infinity` and never reaps children. Harmless; cleared on release.
`does not support tools` (HTTP 400)	An agent attaches a `tools` array to every request and the model has no such capability. Not fixable by configuration — check Table 6 and use a model that declares *tools*, or route the task to an auxiliary slot that is called without them.
Agent reports a rejected key for a provider you never configured	The client fell back to its default provider because the custom one failed to resolve. In Hermes Agent a bare `provider: custom` does exactly this (Section 9); the error names the fallback, not the real fault.
Allocation failure on long conversations	The configured context window exceeds what the KV cache can occupy in the VRAM left after the weights. Lower `context_length`, or claim a second card. See Section 5.5.
`gpu claim` refuses while you hold a card	One claim per user, by design: `"you already hold gpu N -- 'gpu release' first"`. Adding a card means releasing and re-claiming, which destroys the container and every process in it.

Table 12: Symptoms seen in practice, with cause and remedy.

AMERICA'S PRIVATE INDUSTRIAL POWER GRID

12 Command reference

```
# --- pool ------------------------------------------------------------
gpu status
gpu claim 1
gpu shell                       # interactive
gpu shell /path/to/script.sh    # one command
gpu release

# --- inside the container --------------------------------------------
nvidia-smi
ps -eo pid,etime,cmd | grep "[b]in/ollama serve"
tail -f /shared/$USER/ollama-serve.log

# --- model -----------------------------------------------------------
ollama list
ollama pull  qwen3.5:122b-a10b
ollama run   qwen3.5:122b-a10b --verbose "prompt"
ollama stop  qwen3.5:122b-a10b   # unload from VRAM, keep on disk
ollama rm    <model>             # delete weights from /shared

# capabilities decide what a model may be used for - check before wiring
curl -s http://localhost:11435/api/show -d '{"model":"qwen3.5:122b-a10b"}' \
  | python3 -c "import json,sys; print(json.load(sys.stdin)['capabilities'])"

# --- endpoint --------------------------------------------------------
curl http://localhost:11435/api/tags
curl http://localhost:11435/api/ps          # what is resident in VRAM
curl http://localhost:11435/api/generate \
  -d '{"model":"qwen3.5:122b-a10b","prompt":"...","stream":false}'
```

corpus, on the developer VM

```
# --- index -----------------------------------------------------------
./rag ingest                            # new uploads in ~/work/raw
./rag ingest /shared/eugen/brochures    # a folder elsewhere
./rag ingest --force                    # re-parse everything
./rag ingest --reset                    # rebuild from scratch
./rag status                            # what is indexed

# --- ask -------------------------------------------------------------
./rag ask    "..."                      # cited answer
./rag ask    "..." --fast               # skip the rerank call
./rag search "..."                      # passages only

# --- server ----------------------------------------------------------
tmux new -s corpus-mcp
./mcp-http --host 0.0.0.0               # MCP + JSON on :8765, bearer token
./mcp-http --show-client-config

# --- health check ----------------------------------------------------
# No token must return 401. A bare tools/list returns -32600 Missing
# session ID: initialize first, then reuse the mcp-session-id header.
# Full sequence in Section 9.5.
curl -s -o /dev/null -w '%{http_code}\n' -X POST \
  http://127.0.0.1:8765/mcp -H 'Content-Type: application/json' -d '{}'
```